```
#ifndef SORTING_H
#define SORTING_H

#include <iostream>

void displayArray(const int arr[], int size) {
    for (int i = 0; i < size; ++i) {
        std::cout << arr[i] << " ";
    }
}

template <typename T>
void bubbleSort(T arr[], int maxSize){
    for(int i = 0; i < maxSize; i++){
        for(int j = i + 1; j < maxSize; j++){
            if(arr[i] > arr[j]){
                std::swap(arr[i], arr[j]);
            }
        }
    }
}

template <typename T>
int routine_smallest(T arr[], int k, int arrSize){
    int position = k;

    T smallestElement = arr[k];

    for(int j = k + 1; j < arrSize; j++){
        if(arr[j] < smallestElement){
            smallestElement = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], int arrSize){
    int min;

    for(int i = 0; i < arrSize; i++){
        min = routine_smallest(arr, i, arrSize);
        std::swap(arr[i], arr[min]);
    }

}

template <typename T>
void insertionSort(T arr[], int arrSize){
    int k = 1, j;
    T temp;

    while(k < arrSize){
        temp = arr[k];
        j = k - 1;
        while(j >= 0 && temp <= arr[j]){
            arr[j + 1] = arr[j];
            j--;
```

```
        }
        arr[j + 1] = temp;
        k++;
    }
}
#endif
```